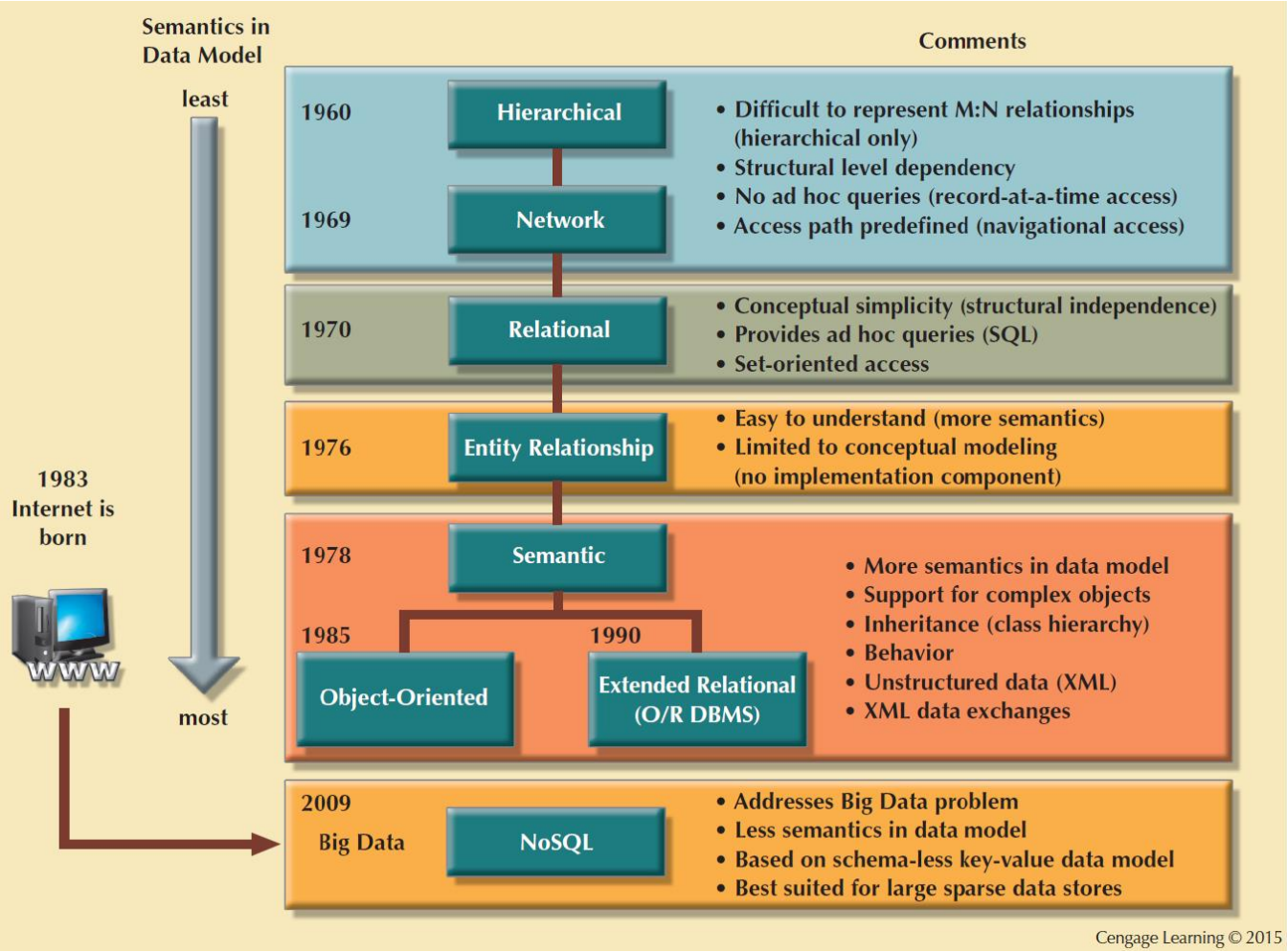
Database Development
158.337

Week 10

1. Chapter 14
2. The Little MongoDB Book by Karl Seguin

https://www.openmymind.net/mongodb.pdf

Figure 2.6 - The Evolution of Data Models



Big Data

Big Data refers to a movement to find new and better ways to manage large amounts of web-generated data and derive business insight from it, while, at the same time, providing high performance and scalability at a reasonable cost.

It comes from multiple and varied sources such as web site logs, web site posts in social sites (LinkedIn, Facebook, Twitter) and machine generated information (GPS, sensors, etc.)

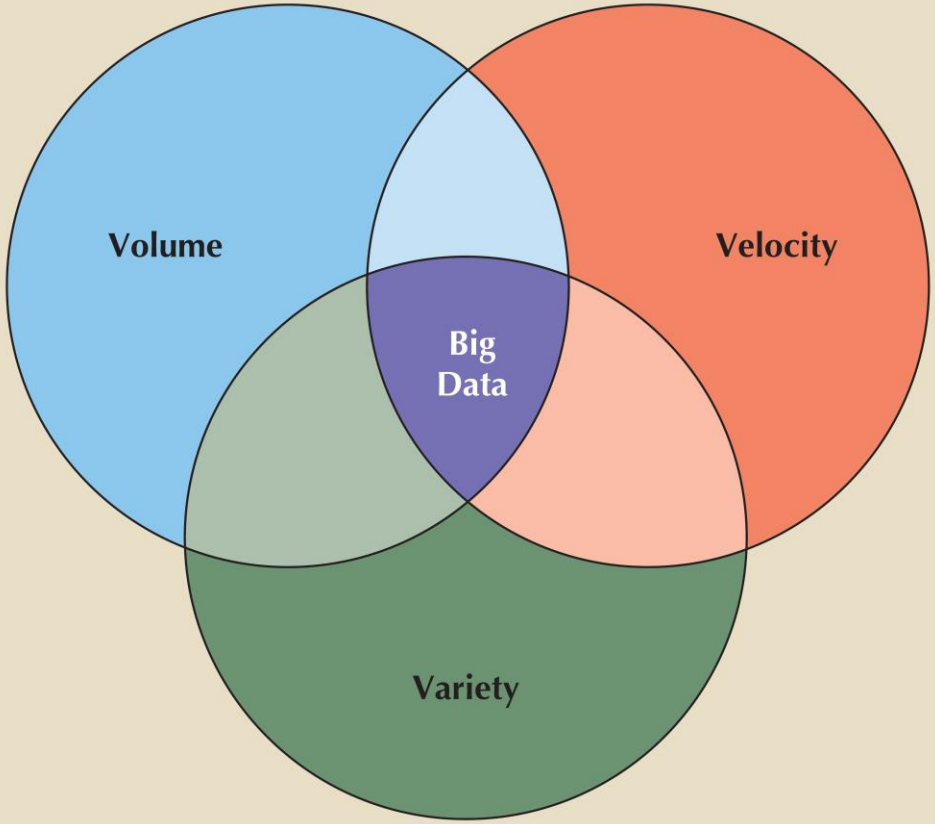
The need to manage and leverage this data has triggered a phenomenon labeled “Big Data”.

Big Data Challenges

- Volume does not allow the usage of conventional structures
- Expensive
- OLAP tools proved inconsistent dealing with unstructured data

Big Data

FIGURE 14.1 ORIGINAL VIEW OF BIG DATA



Big Data

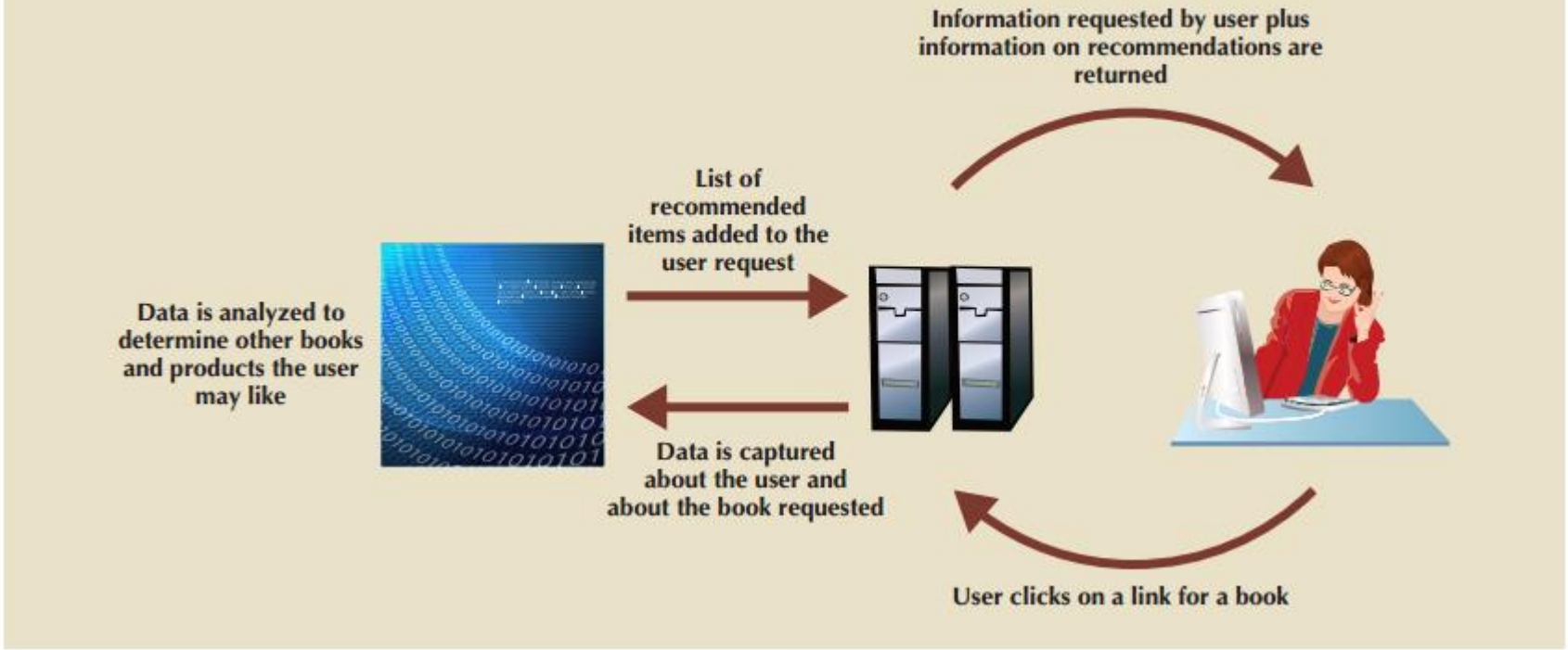
- **Volume:** quantity of data to be stored
  - Scaling up: keeping the same number of systems but migrating each one to a larger system
  - Scaling out: when the workload exceeds server capacity, it is spread out across a number of servers

- **Velocity:** speed at which data is entered into system and must be processed
  - Stream processing: focuses on input processing and requires analysis of data stream as it enters the system
  - Feedback loop processing: analysis of data to produce actionable results

- **Variety:** variations in the structure of data to be stored
  - Structured data: fits into a predefined data model
  - Unstructured data: does not fit into a predefined model

Big Data

FIGURE 14.3 FEEDBACK LOOP PROCESSING



Big Data

- Other characteristics
  - **Variability:** changes in meaning of data based on context
  - Sentimental analysis: attempts to determine if a statement conveys a positive, negative, or neutral attitude about a topic
  - **Veracity:** trustworthiness of data
  - **Value:** degree data can be analyzed for meaningful insight
  - **Visualization:** ability to graphically represent data to make it understandable

- Relational databases are not necessarily the best for storing and managing all organizational data
  - Polyglot persistence: coexistence of a variety of data storage and management technologies within an organization's infrastructure

Big Data New Technologies

Hadoop

Hadoop Distributed File System (HDFS)

MapReduce

NoSQL

Hadoop

Hadoop is a Java based, open source, high speed, fault-tolerant distributed storage and computational framework.

Hadoop uses low-cost hardware to create clusters of thousands of computer nodes to store and process data.

Hadoop originated from Google's work on distributed file systems and parallel processing and is currently supported by the Apache Software Foundation.

HDFS

- Hadoop Distributed File System (HDFS)
  - Based on several key assumptions
    - High volume: default block sizes is 64 MB that can be configured to even larger values (e.g., 128 MB blocks being used in industry)
    - Write-once, read-many: model simplifies concurrency issues and improves data throughput
    - Streaming access: optimized for batch processing of entire files as a continuous stream of data
    - Fault tolerance: designed to replicate data across many different devices so that when one fails, data is still available from another device

Map Reduce

Framework used to process large data sets across clusters

- Breaks down complex tasks into smaller subtasks by performing the subtasks to produce a final result.
- Map function takes a collection of data and sorts and filters it into a set of key-value pairs
  - Mapper program performs the map function
- Reduce summaries results of map function produce a single result
  - Reducer program performs the reduce function

Implementation complements HDFS structure

- Job tracker: central control program
- Task tracker: reduces tasks on a node
- Batch processing: runs tasks from beginning to end with no user interaction

NoSQL Databases

- Not based on the relational database structure
- Not Only SQL
- Not characterized by ACID properties -- Atomicity, Consistency, Isolation and Durability
- It is characterized by BASE –
  - Basically Available (i.e., failures may occur)
  - Soft state (i.e., Data consistency is not a hard requirement)
  - Eventually consistent (i.e., at some future point, the data will assume a consistent state)

ACID Properties

Atomicity

- All operations of a transaction must be completed. If not, the transaction is aborted

Consistency

- Permanence of database's consistent state

Isolation

- Data used during transaction cannot be used by second transaction until the first is completed

Durability

- Ensures that once transactions are committed, they cannot be undone or lost

Serializability

- Ensures that the schedule for the concurrent execution of several transactions should yield consistent results

BASE Properties

Basically Available

- The availability of data even in the presence of multiple failures - If a single node fails, part of the data won't be available, but the entire data layer stays operational.

Soft State

- Data consistency is not a hard requirement. Data consistency is the developer's problem and should not be handled by the database

Eventually Consistent

- At some point in the future, data will converge to a consistent state.

NoSQL Databases

- Support distributed database architectures
- Provide high scalability, high availability, and fault tolerance
- Support large amounts of sparse data
- Geared toward performance rather than transaction consistency
- Four categories: key-value data stores, document databases, column-oriented databases and graph databases

No SQL

Advantages

- High scalability, availability, and fault tolerance are provided
- Uses low-cost commodity hardware
- Supports Big Data
- Key-value model improves storage efficiency

Disadvantages

- Complex programming is required
- There is no relationship support
- There is no transaction integrity support
- In terms of data consistency, it provides an eventually consistent model

Sparse Data

- Sparse data refers to cases in which the number of attributes are very large, but the actual number of distinct value instances is relatively small.

DID	CERT1	CERT2	CERT3	DOB	LICTYPE
2732	80		95	1/24/1962	P
2946		92		4/11/1970	
3650	86			11/27/1963	R

Sparse Data – Census Data

Census data – has an entity called person.

- A person can have hundred of attributes, some of those attributes would be first name, last name, degree, employer, income, veteran status, foreign born, etc.
- Although, there would be many millions of rows of data for each person, there will be many attributes that will be left blank,
  - not all persons will have a degree, an income or an employer.
  - Even fewer persons will be veterans or foreign born.

Databases: KV & Document

- Key-value (KV) databases: conceptually the simplest of the NoSQL data models
  - Store data as a collection of key-value pairs organized as buckets which are the equivalent of tables
- Document databases: are similar to key-value databases and can almost be considered a subtype of KV databases
  - Store data in key-value pairs in which the value components are encoded documents grouped into large groups called collections

Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"

Collection = Customer

Key	Document
10010	{LName: "Ramas", FName: "Alfred", Initial: "A", Areacode: "615", Phone: "844-2573", Balance: "0"}
10011	{LName: "Dunne", FName: "Leona", Initial: "K", Areacode: "713", Phone: "894-1238", Balance: "0"}
10014	{LName: "Orlando", FName: "Myron", Areacode: "615", Phone: "222-1672", Balance: "0"}

Database Shard

Traditional database design – Table form (i.e., We split data vertically into many columns).

Well, what if we split them horizontally?

> Here, the row is now split, such that we have multiple rows for the same entity occurrence. Each partition forms part of a **shard**, which may in turn be located on a separate database server or physical location.

So, a search load for the large partitioned table can be split across multiple servers, not just multiple indexes on the same logical server.

FIGURE 2.5 A simple key-value representation

Trucks-R-Us

Data stored using traditional relational model

DID	CERT1	CERT2	CERT3	DOB	LICTYPE
2732	80		95	1/24/1962	P
2946		92		4/11/1970	
3650	86			11/27/1963	R

Data stored using key-value model

DID	KEY	VALUE
2732	CERT1	80
2732	CERT3	95
2732	DOB	1/24/1962
2732	LICTYPE	P
2946	CERT2	92
2946	DOB	4/11/1970
3650	CERT1	86
3650	DOB	11/27/63
3650	LICTYPE	R



Driver 2732

- In the relational model:
  - Each row represents one entity instance
  - Each column represents one attribute of the entity
  - The values in a column are of the same data type
- In the key-value model:
  - Each row represents one attribute/value of one entity instance
  - The "key" column could represent any entity's attribute
  - The values in the "value" column could be of any data type and therefore it is generally assigned a long string data type

SOURCE: Course Technology/Cengage Learning

DID	CERT1	CERT2	CERT3	DOB	LICTYPE
2732	80		95	1/24/1962	P
2946		92		4/11/1970	
3650	86			11/27/1963	R

Shard it

DID	KEY	VALUE
2732	CERT1	80
2732	CERT3	95
2732	DOB	1/24/1962
2732	LICTYPE	P
2946	CERT2	92
2946	DOB	4/11/1970
3650	CERT1	86
3650	DOB	11/27/63
3650	LICTYPE	R

Key - Value

- The ***key-value*** data model is based on a structure composed of two data elements: a key and a value; in which for every key there is a corresponding value (or a set of values).
- The key-value data model is also referred to as the attribute-value or associative data model. In the key-value data model, each row represents one attribute of an entity occurrence.
  - The "key" column points to an attribute
  - The "value" column contains the actual value for the attribute. The data type of the "value" column is generally a long string to accommodate the variety of actual data types of the values that are placed in the column.

Emerging Data Models: Big Data and NoSQL (cont'd.)

- Key-value data model
  - Two data elements: key and value
    - Every key has a corresponding value or set of values
- Sparse data
  - Number of attributes is very large
  - Number of actual data instances is low
- Eventual consistency
  - Updates will propagate through system; eventually all data copies will be consistent

NoSQL – Data Model

Offers users options as to the level of consistency they want. i.e. users can decide –

I want it totally consistent at the cost of high latency

OR

I will compromise on consistency, but I want fast response times.

Levels of consistency

- Absolute consistency
- Time-based consistency
- Version-based consistency

NoSQL DBMS

NoSQL databases are used in applications which

- allow high concurrent read and write performance
- can efficiently store and access huge amounts of data
- have high scalability and high availability
- are highly optimized for retrieve and append operations but offer little functionality beyond record storage
- have high latency or lag (i.e. low latency means fast response time)
- characterized by BASE acronym

Relational versus Big Data

- ACID versus BASE
- Structured versus Unstructured
- Rows versus Complex Objects
- Joins/Relationships versus Physical Mapping
- Group By versus Map Reduce
- Concurrency versus Random/Read only/ Clumsy/ non-concurrent
- Transaction Oriented versus Social-Network
- Logical Schema versus Key-Value
- Centralized versus Clusters

MongoDB

The name, MongoDB, comes from the word humongous as its developers intended their new product to support extremely large data sets.

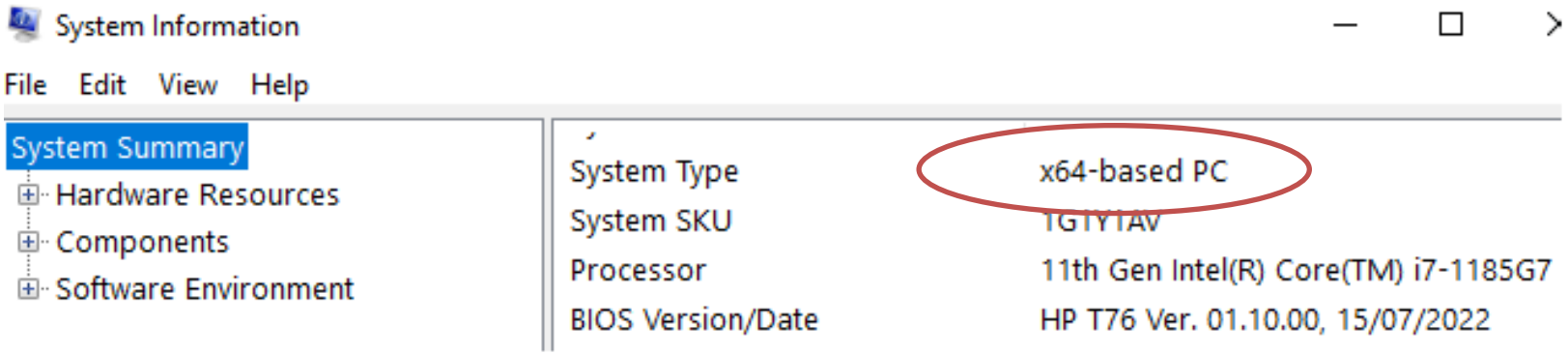
MongoDB is an open source document-oriented NoSQL database which stores data in the form of JSON-like objects.

It has emerged as one of the leading databases due to its dynamic schema, high scalability, optimal query performance, faster indexing and an active user community.

NOTE: JSON is JavaScript Object Notation (an alternative to XML)

Using MongoDB on VMWare or in your home machine

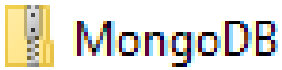
- Check your system type.
- MongoDB requires x86-64 architecture



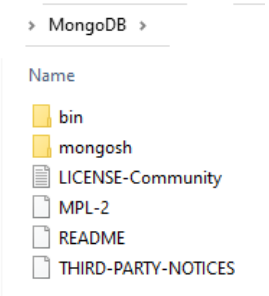
Using MongoDB on VMWare or in your home machine

Steps

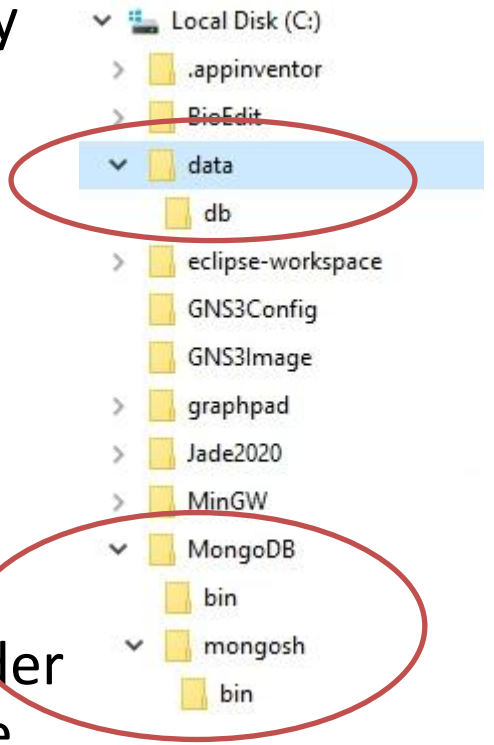
1. A zipped folder (which has the required binary files) is provided on Stream.



2. Unzip and copy this folder into C: drive (C:\MongoDB)

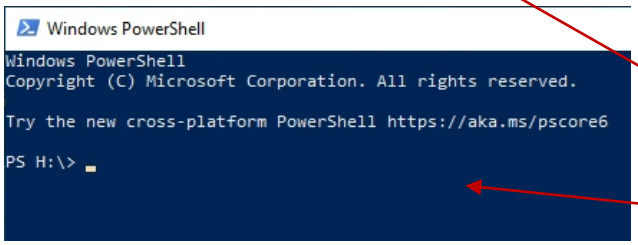


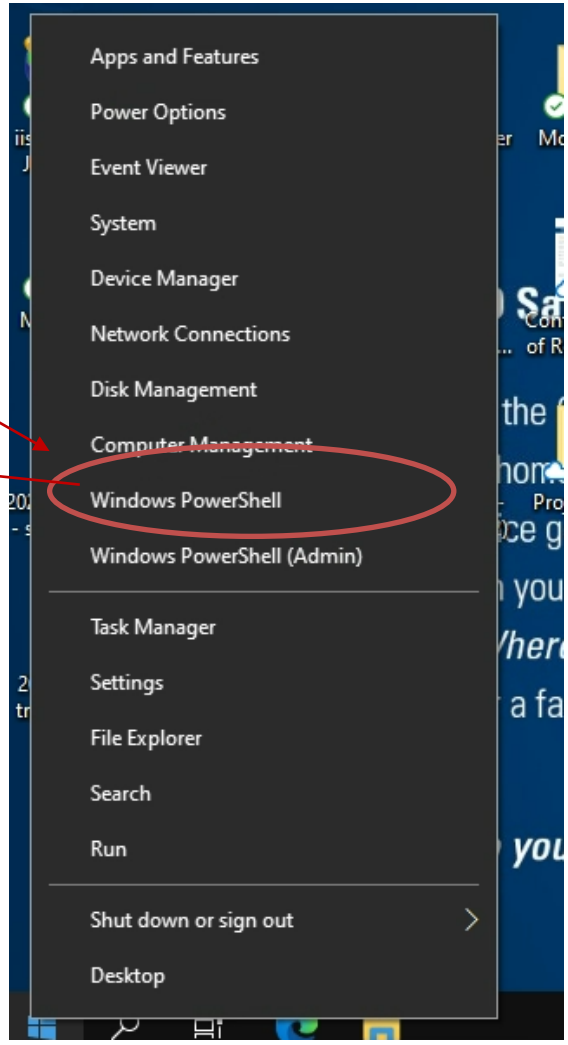
3. Create a folder data in C: drive and then a folder db (path C:\data\db must exist). This is where your collections will be stored.



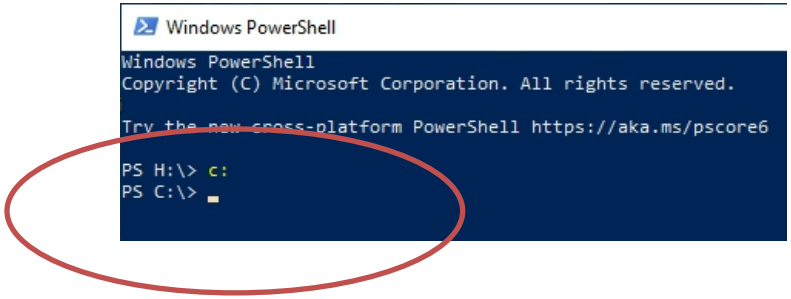
Using MongoDB on VMWare or in your home machine

4. Open Windows PowerShell



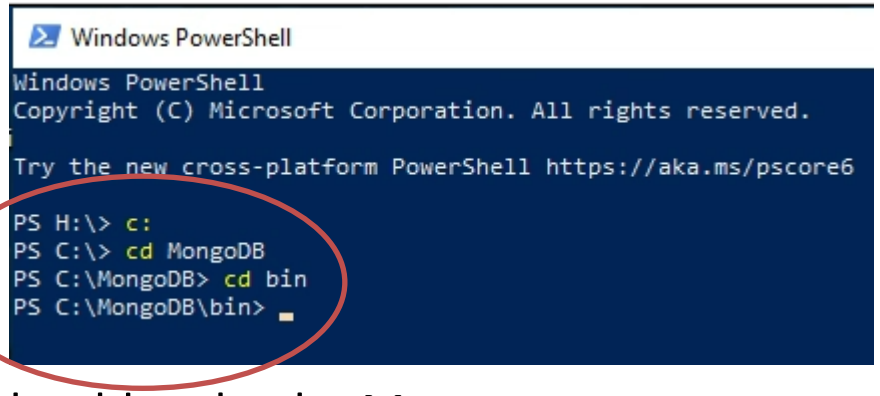


5. Navigate to C: drive

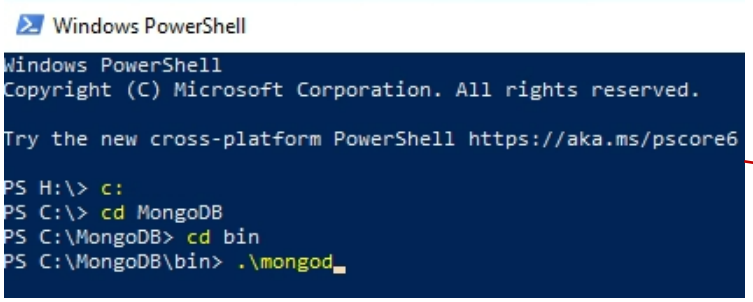


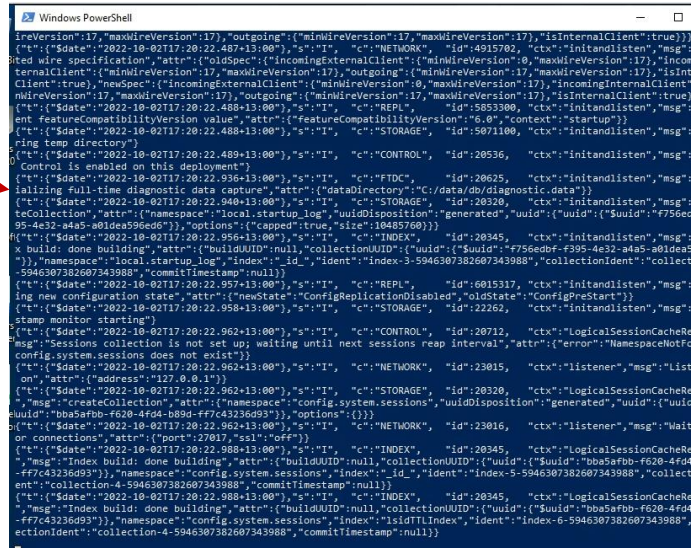
Using MongoDB on VMWare or in your home machine

6. Navigate to C:\MongoDB\bin



7. Enter the command .\mongod. This should make the Mongo server run.





Leave this open. Next, we will open the Mongo shell

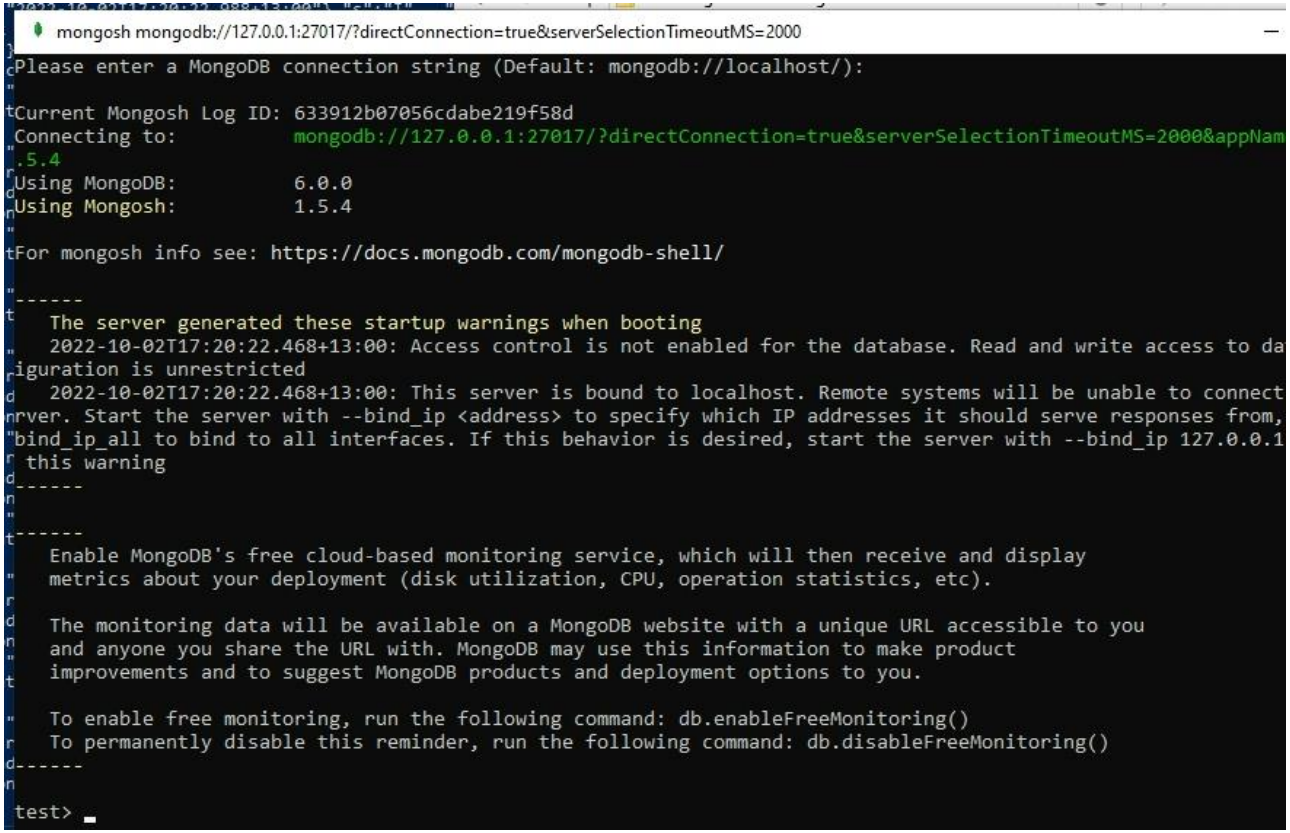
Using MongoDB on VMWare or in your home machine

6. From File Explorer, go to C:\MongoDB\mongosh\bin and double click mongosh application. This will open a command prompt window:



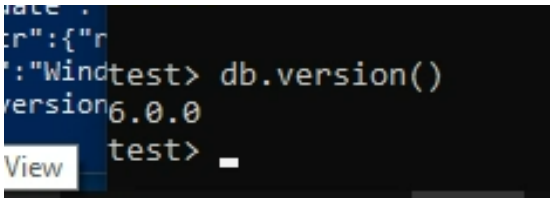
You are set. Press enter for default server setting.

This will open the mongo shell with test>

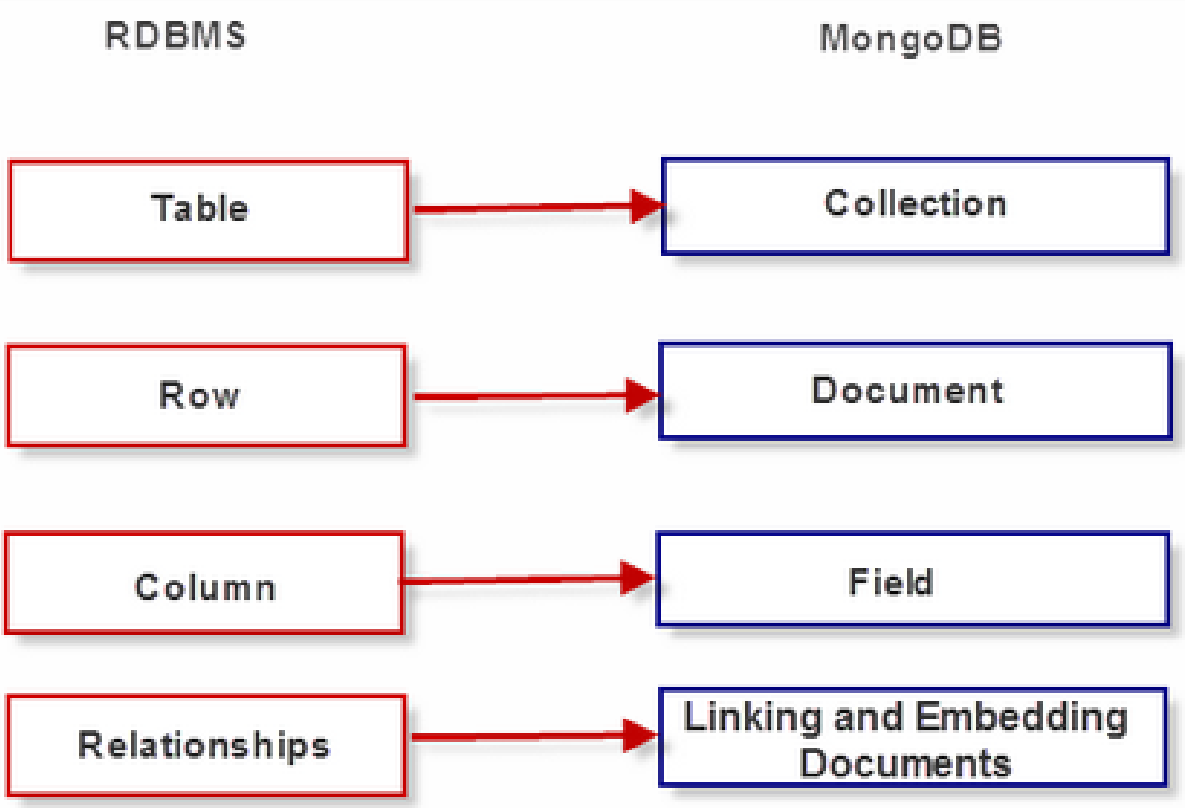


Type db.version() and you should get 6.0.0





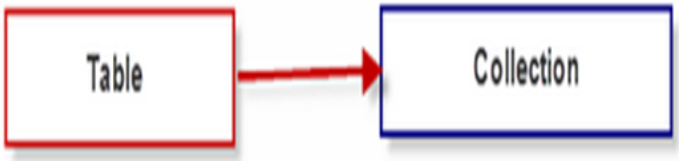
RDBMS versus MongoDB concepts



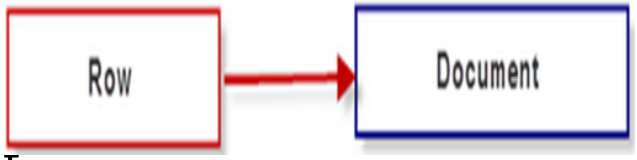
http://code.tutsplus.com/articles/mapping-relational-databases-and-sql-to-mongodb--net-35650

RDBMS versus MongoDB concepts

Since collections are schema-less, we don't explicitly need to create them



When we create a document, the collection is also created.



Let us create a document

Name	Sex	Weight
Aurora	f	450

```
db.unicorns.insert({name: "Aurora", gender: "f", weight: 450});
```

See collections

```
db.getCollectionNames();
```

Drop collections

```
db.unicorns.drop();
```

```
test> db.getCollectionNames();
[ 'unicorns' ]
test> db.unicorns.drop()
true
test> db.getCollectionNames();
[]
test>
```

Select statement

db.unicorns.insert({name: "Aurora", gender: "f", weight: 450});

```
test> db.unicorns.insert({name: "Aurora", gender: "f", weight: 450});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("63391fca5dc9a314e205916e") }
}
```

db.unicorns.find();

```
test> db.unicorns.find();
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
test>
```

db.unicorns.find().pretty();

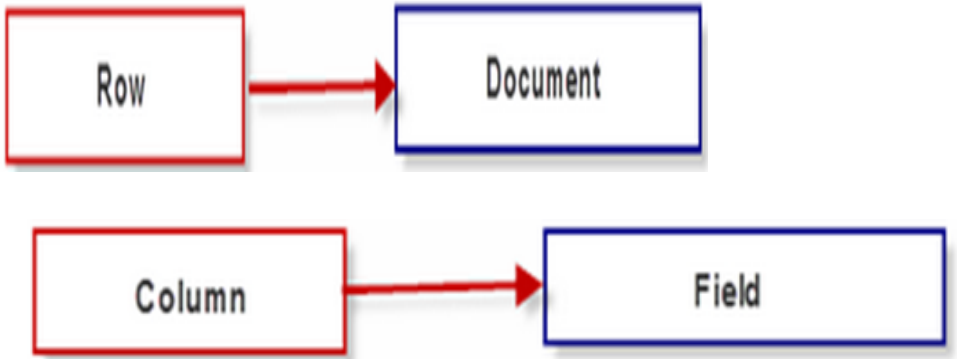
```
test> db.unicorns.find().pretty();
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
```

RDBMS versus MongoDB concepts

db.unicorns.find();

Will show four fields:

_id, name., gender, weight



Note the _id field. Every document must have a unique _id field

```
test> db.unicorns.find().pretty();
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
```

Alternatively, you could also specify your own ObjectId

Best to let MongoDB create the id for you (since Big Data knows its job).

RDBMS versus MongoDB concepts

Add another document (or row)

Name	Sex	Weight	home	worm
Aurora	f	450		
Leto	m		Arrakeen	false



db.unicorns.insert({name: "Leto", gender: "m", home: "Arrakeen", worm: false})
db.unicorns.find()

```
test> db.unicorns.insert({name: "Leto", gender: "m", home: "Arrakeen", worm: false});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("633921b05dc9a314e205916f") }
}
test> db.unicorns.find();
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("633921b05dc9a314e205916f"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false
  }
]
```

JSON Documents

Now, when we need to embed a list of things (arrays/collections) inside a document, we use JSON



Example:

Unicorn John has interest – boxing, fishing, swimming, playing rugby. Then with JSON, we can use subdocuments

```
db.unicorns.insert({name:"John", gender: "m", home: "Auckland", interests:["swimming", "Boxing", "fishing", "playing rugby"]});
```

```
test> db.unicorns.insert({name:"John",gender: "m", home:"Auckland", interests:["swimming", "B
oxing", "fishing", "playing rugby"]});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("6339224c5dc9a314e2059170") }
}
```

JSON Documents

Imagine if this was in a structured table….

Aurora: 4 attributes; Leto: 5 attributes; John: 5 attributes of which one is multi-valued

```
test> db.unicorns.find();
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("633921b05dc9a314e205916f"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false
  },
  {
    _id: ObjectId("6339224c5dc9a314e2059170"),
    name: 'John',
    gender: 'm',
    home: 'Auckland',
    interests: [ 'swimming', 'Boxing', 'fishing', 'playing rugby' ]
  }
]
```

JSON Documents

Using the filter clause

```
db.unicorns.find({gender: 'f', weight: {$gt: 300}});
```

OR

```
db.unicorns.find({gender: {$ne:'m'}, weight: {$gt: 300}})
```

```
test> db.unicorns.find({gender: 'f', weight: {$gt: 300}});
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
test> db.unicorns.find({gender: {$ne:'m'}, weight: {$gt: 300}})
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
```

JSON Documents

Use more queries:

```
test> db.unicorns.find({vampires: {$exists: false}});
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("633921b05dc9a314e205916f"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false
  },
  {
    _id: ObjectId("6339224c5dc9a314e2059170"),
    name: 'John',
    gender: 'm',
    home: 'Auckland',
    interests: [ 'swimming', 'Boxing', 'fishing', 'playing rugby' ]
  },
  {
    _id: ObjectId("633924b15dc9a314e205917a"),
    name: 'Nimue',
    dob: ISODate("1999-12-20T03:15:00.000Z"),
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

JSON Documents

Update one unicorn

db.collection.updateOne (filter, update, options)

Update Roooooodles

```
test> db.unicorns.find({name:'Roooooodles'});
[
  {
    _id: ObjectId("633924b15dc9a314e2059174"),
    name: 'Roooooodles',
    dob: ISODate("1979-08-18T06:44:00.000Z"),
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  }
```

UpdateOne:

```
test> db.unicorns.updateOne({name: "Roooooodles"}, {$set: {weight: 590}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
test> db.unicorns.find({name: "Roooooodles"});
[
  {
    _id: ObjectId("633924b15dc9a314e2059174"),
    name: 'Roooooodles',
    dob: ISODate("1979-08-18T06:44:00.000Z"),
    loves: [ 'apple' ],
    weight: 590,
    gender: 'm',
    vampires: 99
  }
]
```

UpdateOne

Update the unicorn Pilot

```
{
  _id: ObjectId("633924b15dc9a314e2059179"),
  name: 'Pilot',
  dob: ISODate("1997-02-28T16:03:00.000Z"),
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
```

```
test> db.unicorns.updateOne({name: "Pilot"}, {$inc: {vampires: -2}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.unicorns.find({name: 'Pilot'});
[
  {
    _id: ObjectId("633924b15dc9a314e2059179"),
    name: 'Pilot',
    dob: ISODate("1997-02-28T16:03:00.000Z"),
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 52
  }
]
```

UpdateMany

Update many unicorns

db.collection.updateMany(filter, update, options)

I have two Unicorns by the name Aurora

```
test> db.unicorns.find({name: 'Aurora'});
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("633924b15dc9a314e2059172"),
    name: 'Aurora',
    dob: ISODate("1991-01-24T00:00:00.000Z"),
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

UpdateMany

Both unicorns with the name Aurora love sugar

```
test> db.unicorns.updateMany({name: 'Aurora'}, {$push: {loves: 'sugar'}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
test> db.unicorns.find({name: 'Aurora'});
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450,
    loves: [ 'sugar' ]
  },
  {
    _id: ObjectId("633924b15dc9a314e2059172"),
    name: 'Aurora',
    dob: ISODate("1991-01-24T00:00:00.000Z"),
    loves: [ 'carrot', 'grape', 'sugar' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

```
test> db.unicorns.updateMany({}, {$set: {vaccinated: true }});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 14,
  modifiedCount: 14,
  upsertedCount: 0
}
```

```
test> db.unicorns.find({vaccinated: true});
[
  {
    _id: ObjectId("63391fca5dc9a314e205916e"),
    name: 'Aurora',
    gender: 'f',
    weight: 450,
    loves: [ 'sugar' ],
    vaccinated: true
  },
  {
    _id: ObjectId("633921b05dc9a314e205916f"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false,
    vaccinated: true
  },
  {
    _id: ObjectId("6339224c5dc9a314e2059170"),
    name: 'John',
    gender: 'm',
    home: 'Auckland',
    interests: [ 'swimming', 'Boxing', 'fishing',
    vaccinated: true
  },
  {
    _id: ObjectId("633924b15dc9a314e2059171"),
    name: 'Honey',
    dob: ISODate("1992-03-12T18:47:00.000Z"),
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63,
    vaccinated: true
```

Using find

```
test> db.unicorns.find(null, {name: 1});
[
```

```
test> db.unicorns.find(null, {name: 1, _id:0});
[
```

```
test> db.unicorns.find().sort({weight: -1});
```

```
test> db.unicorns.find().sort({name: 1, vampires: -1});
```

```
test> db.unicorns.find({vampires: {$gt: 50}}).count();
5
```

```
test> db.unicorns.aggregate([{$group:{_id:'$gender',total: {$sum:1}}}]);
[ { _id: 'f', total: 5 }, { _id: 'm', total: 9 } ]
```

Using find

```
test> db.unicorns.aggregate([{$unwind:'$loves'},
... {$group: {_id:'$loves', total:{$sum:1}, unicorns:{$addToSet:'$name'}}},
... {$sort:{total:-1}},
... {$limit:1} ]);
[
  {
    _id: 'grape',
    total: 4,
    unicorns: [ 'Kenny', 'Dunx', 'Nimue', 'Aurora' ]
  }
]
```

```
test> db.unicorns.aggregate([{$unwind:'$loves'},
... {$group: {_id:'$loves', total:{$sum:1}, unicorns:{$addToSet:'$name'}}},
... {$sort:{total:1}},{$limit:5} ]);
[
  { _id: 'energon', total: 1, unicorns: [ 'Unicrom' ] },
  { _id: 'papaya', total: 1, unicorns: [ 'Honey' ] },
  { _id: 'redbull', total: 1, unicorns: [ 'Unicrom' ] },
  { _id: 'strawberry', total: 1, unicorns: [ 'Ayna' ] },
  { _id: 'lemon', total: 2, unicorns: [ 'Kenny', 'Ayna' ] }
]
```

To Embed or NOT to Embed?

1. To embed or NOT to embed depends upon the way you access the data that guides it.

2. Also, if you have more than 16MB data, then best to put in a separate collection, and NOT to embed it.

3. Linking Documents

Employees

db.employees.insert({_id: ObjectId("4d85c7039ab0fd70a117d730"), name: 'Leto'});

db.employees.insert({_id: ObjectId("4d85c7039ab0fd70a117d731"), name: 'Duncan', manager: ObjectId ("4d85c7039ab0fd70a117d730")});

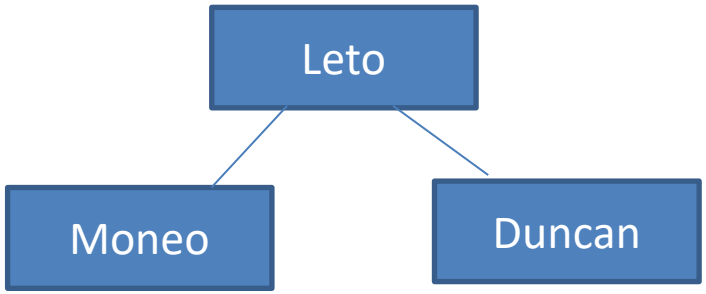
db.employees.insert({_id: ObjectId("4d85c7039ab0fd70a117d732"), name: 'Moneo', manager: ObjectId("4d85c7039ab0fd70a117d730")});

```
test> db.employees.find();
[
  { _id: ObjectId("4d85c7039ab0fd70a117d730"), name: 'Leto' },
  {
    _id: ObjectId("4d85c7039ab0fd70a117d731"),
    name: 'Duncan',
    manager: ObjectId("4d85c7039ab0fd70a117d730")
  },
  {
    _id: ObjectId("4d85c7039ab0fd70a117d732"),
    name: 'Moneo',
    manager: ObjectId("4d85c7039ab0fd70a117d730")
  }
]
```

Employees

db.employees.find({manager: ObjectId("4d85c7039ab0fd70a117d730")});

```
test> db.employees.find({manager: ObjectId("4d85c7039ab0fd70a117d730")});
[
  {
    _id: ObjectId("4d85c7039ab0fd70a117d731"),
    name: 'Duncan',
    manager: ObjectId("4d85c7039ab0fd70a117d730")
  },
  {
    _id: ObjectId("4d85c7039ab0fd70a117d732"),
    name: 'Moneo',
    manager: ObjectId("4d85c7039ab0fd70a117d730")
  }
]
```




```
test> db.getCollectionNames();
[ 'unicorns', 'employees' ]
```

Data Modelling by Linking

```
db.masseyStaff.insert({_id: 121, name: "Michael Ahie", position: "Chancellor"});
db.masseyStaff.insert({_id: 122, name: "Jan Thomas", position: "Vice Chancellor",
manager:{_id:121}});
db.masseyStaff.insert({_id: 123, name: "Ray Geor", position: "Pro Vice Chancellor",
manager:{_id:122}});
db.masseyStaff.insert({_id: 124, name: "Chris Scogings", position: "Head of School",
manager:{_id:123}});
db.masseyStaff.insert({_id: 125, name: "Anu Mathrani", position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 126, name: "Binglan Han", position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 127, name: "Teo Susnjak",  position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 128, name: "Andre Barcak", position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 129, name: "Daniel Playne", position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 130, name: "Indu Sofat", position: "Academic staff",
manager:{_id:123}});
db.masseyStaff.insert({_id: 131, name: "XYZ XYZ", position: "Admin Lead",
manager:{_id:122}});
db.masseyStaff.insert({_id: 132, name: "ABC ABC", position: "Admin staff",
manager:{_id:131}});
```

```
test> db. getCollectionNames();
[ 'masseyStaff', 'unicorns', 'employees' ]
```

Data Modelling by Linking

Say an employee has two managers:

```
db.masseyStaff.insert({_id: 133, name: "Haahaa Hoohoo", position: "Finance/Admin staff", manager:[{_id:122}, {_id:123}]});
```

Really cool way to demonstrate many-to-many joins

1. A staff may have many reporting staff
2. A staff may report to many staff

```
db.masseyStaff.find({$and:[{manager:{_id:122}},{manager:{_id:123}}]});
```

```
test> db.masseyStaff.find({$and:[{manager:{_id:122}},{manager:{_id:123}}]});
[
  {
    _id: 133,
    name: 'Haahaa Hoohoo',
    position: 'Finance/Admin staff',
    manager: [ { _id: 122 }, { _id: 123 } ]
  }
]
```

db.masseyStaff.find();



Find with Filter Option

```
db.masseyStaff.find({manager:{_id:121}});
db.masseyStaff.find({manager:{_id:122}}));
db.masseyStaff.find({manager:{_id:123}});
db.masseyStaff.find({manager:{_id:123}},{name:1, _id:0, manager:1});
```

Aggregate

```
masseyStaff.aggregate([{$group:{_id:'$manager',total: {$sum:1}}}]);
```

```
test> db.masseyStaff.aggregate([{$group:{_id:'$manager',total: {$sum:1}}}]);
[
  { _id: { _id: 123 }, total: 7 },
  { _id: { _id: 121 }, total: 1 },
  { _id: null, total: 1 },
  { _id: { _id: 122 }, total: 2 },
  { _id: [ { _id: 122 }, { _id: 123 } ], total: 1 },
  { _id: { _id: 131 }, total: 1 }
]
```

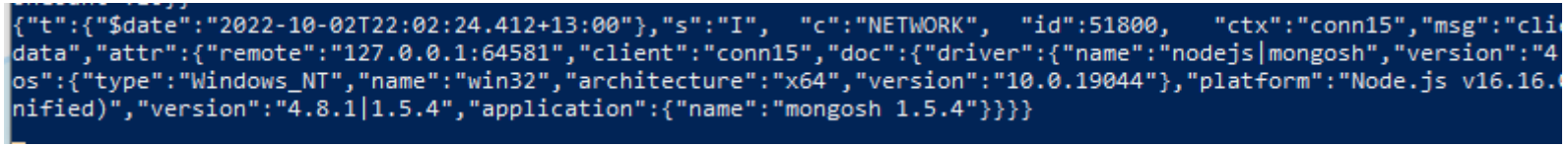
Aggregate by groups

db.masseyStaff.aggregate([{$unwind:'$manager'},{$group: {_id:'$manager', total:{$sum:1}, employees:{$addToSet:'$name'}}},{$sort:{total:-1}},{$limit:3}]);

```
test> db.masseyStaff.aggregate([{$unwind:'$manager'},{$group: {_id:'$manager', total:{$sum:1}
, employees:{$addToSet:'$name'}}},
... {$sort:{total:-1}},
... {$limit:3} ]);
[
  {
    _id: { _id: 123 },
    total: 8,
    employees: [
      'Daniel Playne',
      'Chris Scogings',
      'Anu Mathrani',
      'Binglan Han',
      'Andre Barcak',
      'Haahaa Hoohoo',
      'Indu Sofat',
      'Teo Susnjak'
    ]
  },
  {
    _id: { _id: 122 },
    total: 3,
    employees: [ 'XYZ XYZ', 'Ray Geor', 'Haahaa Hoohoo' ]
  },
  { _id: { _id: 121 }, total: 1, employees: [ 'Jan Thomas' ] }
]
```

Multi-user

- Open two shells

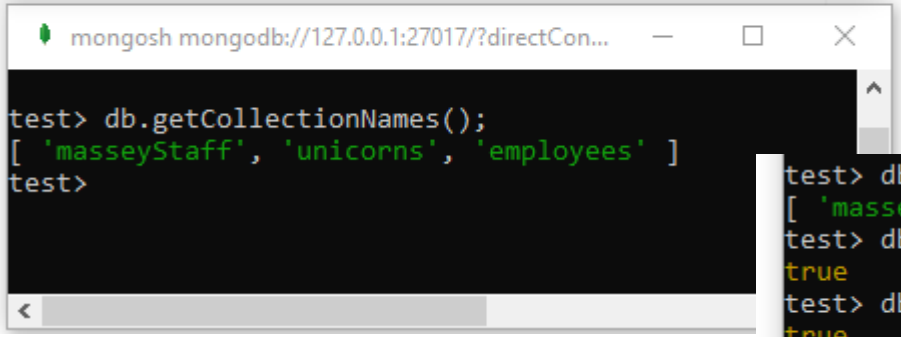


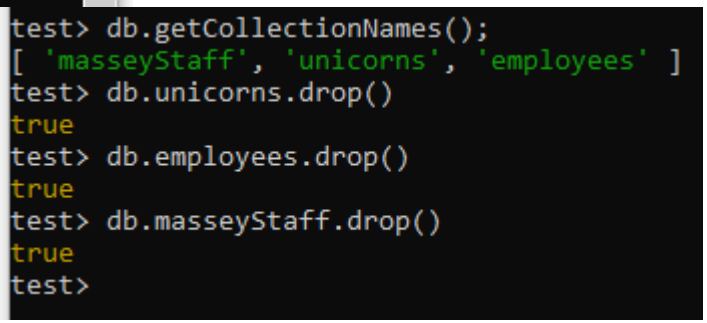
You can access documents from both shells

Shell1



Shell2





Denormalization

- What should you do: embed arrays of ids and do manual referencing or embed documents?
- Embedded documents are frequently used, but mostly for smaller pieces of data which we want to always pull with the parent document.
- The Little MongoDB Book by Karl Seguin
  Available at http://openmymind.net/mongodb.pdf

Questions

1. Two Oracle database administrators went to a NoSQL bar for a night out. After a few minutes they came out. WHY???
2. A SQL statement walks into a bar and sees two tables. He walks up and says…
3. A NoSQL statement walks into a bar and sees two tables. He walks up and says…

Tutorial Exercise

- Install MongoDB
- Create a *Table* (*Collection*)
- Insert a *row* (*Document*)
- Write simple *Select* query (*find()* method)
- Write *Select* with *Where* clause (*find()* with condition)
- Update a *row/document*
- Count *rows/documents*
- Compare SQL queries with NoSQL queries